Project Proposal and Methodology

Hybrid RAG & Fine-Tuning for Customer Support

Piyush Sharma · Executive PG Diploma in Data Science & AI, IIIT Bangalore (via upGrad) · October 2026

This document covers Task 1.1 (project scope) and Task 1.4 (methodology), and reflects the pipeline as implemented in the final notebooks.

1. Project scope (Task 1.1)

1.1 Target use case

The use case is refund-related customer support. Refund queries are frequent, governed by a clear written policy (refund_policy.md) and easy to judge, which makes them a good anchor for intent extraction followed by policy-grounded retrieval. In the Bitext dataset the REFUND category has three intents (check_refund_policy, get_refund and track_refund), which together make up 450 of the 4,000 sampled rows.

The intent router is trained and evaluated over the full Bitext label space (27 intents in 11 categories) rather than on refund queries alone, so that it must separate refund requests from neighbouring intents such as order tracking and invoices. The knowledge base is 13 corporate SOP documents covering refunds and related topics.

1.2 System overview

The system turns a raw, noisy customer query into a structured intent and a policy-grounded response, built as three progressively enhanced pipelines that are evaluated on the same data:

- **Baseline.** The pre-trained Qwen2.5-1.5B-Instruct model with no retrieval and no fine-tuning.
- **Solution V1 (Naive RAG).** The same model, with the top SOP retrieved from ChromaDB using the raw query and injected into the prompt.
- **Solution V2 (Hybrid RAG).** A LoRA fine-tuned intent router extracts a JSON intent first; that intent drives retrieval, and the final response is generated with the retrieved SOP as context.

1.3 Expected JSON input and output

Input (raw customer query, as stored in the instruction column): *i want to check the status of my refund*

Target output (strict JSON, built with json.dumps from the Bitext intent and category columns):

{"intent": "track_refund", "category": "REFUND"}

The intent is the fine-grained Bitext label in lowercase (27 values, for example cancel_order or get_refund). The category is the broader group in uppercase (11 values, for example ORDER or REFUND).

1.4 Project boundaries

In scope	Out of scope
Intent and category extraction from single-turn English customer queries. SOP retrieval with ChromaDB over 13 Markdown documents. A three-way comparison of Baseline, Solution V1 and Solution V2 on the full held-out test split (391 rows), plus an adversarial subset.	Per-category result breakdowns, including a refund-only breakdown. Multi-turn conversations and languages other than English. Production deployment or API integration, and models larger than 1.5 billion parameters.

1.5 Success criteria (Task 1.1.2)

The following measurable targets were set at the proposal stage and are compared with the results in the Comparative Analysis Report.

Metric	Definition	Target
Format adherence	Share of outputs that parse as valid JSON. For V2 it is measured on the final response, and router-only adherence is reported separately	≥ 85%
Intent accuracy	Exact match between the extracted intent and the Bitext label	≥ 80%
ROUGE-1	Unigram overlap with the SOP-grounded reference	≥ 0.35
ROUGE-L	Longest-common-subsequence overlap with the reference	≥ 0.30
BLEU	N-gram precision against the reference	≥ 0.20
Consistency	Identical output across repeated greedy runs	≥ 90%
Hallucination	Policy violations or unsupported facts compared with the SOP	≥ 30% reduction from Baseline to V2

2. Data understanding summary

Findings from the exploratory analysis that shaped the design choices below.

Aspect	Finding
Source	bitext/Bitext-customer-support-llm-chatbot-training-dataset: 26,872 rows with columns flags, instruction, category, intent and response; no missing values.
Working sample	4,000 rows (seed 42) covering 27 intents and 11 categories; intent counts range from 115 to 169 (a 1.5× ratio), so classes are reasonably balanced.
Data quality	83 exact and 15 near-duplicate prompts (98 in total) are removed by lowercase-and-strip deduplication, leaving 3,902 rows. 79 prompts contain double spaces and 938 (23%) contain template placeholders such as {{Order Number}}. Every intent maps to exactly one category.
Prompt length	Mean 10.2 tokens and maximum 21 with the Qwen tokenizer. Full ChatML sequences are at most 78 tokens, so a maximum length of 128 causes no truncation.
Splits	80/10/10 stratified by intent: 3,121 train, 390 validation and 391 test rows, with zero overlap confirmed by set-intersection checks.
SOP corpus	13 Markdown files: refund_policy, shipping_delays, password_reset, technical_troubleshooting, escalation_matrix, order_tracking, billing_disputes, product_return, data_privacy, subscription_cancellation, account_recovery, payment_methods and working_hours. TF-IDF cosine similarity between documents is low (highest 0.42 for password_reset and account_recovery, then 0.38 for order_tracking and shipping_delays), so the documents are topically distinct.

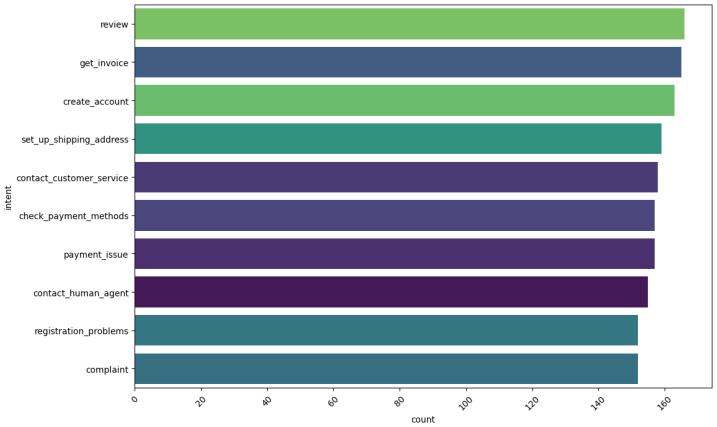


Figure 1. Intent distribution of the 4,000-row sample (ten most frequent intents), from the EDA notebook.

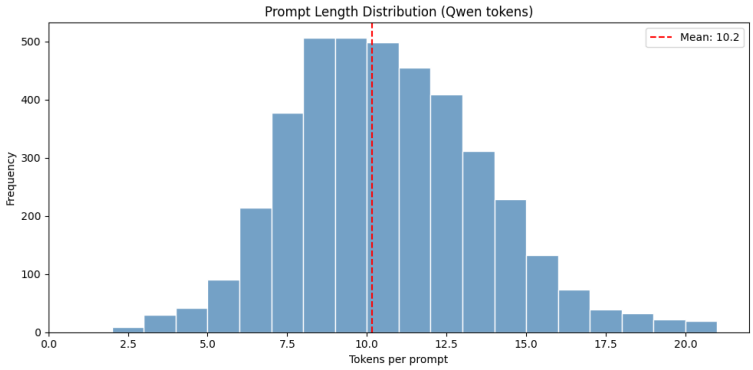


Figure 2. Prompt length distribution in Qwen tokens, from the EDA notebook.

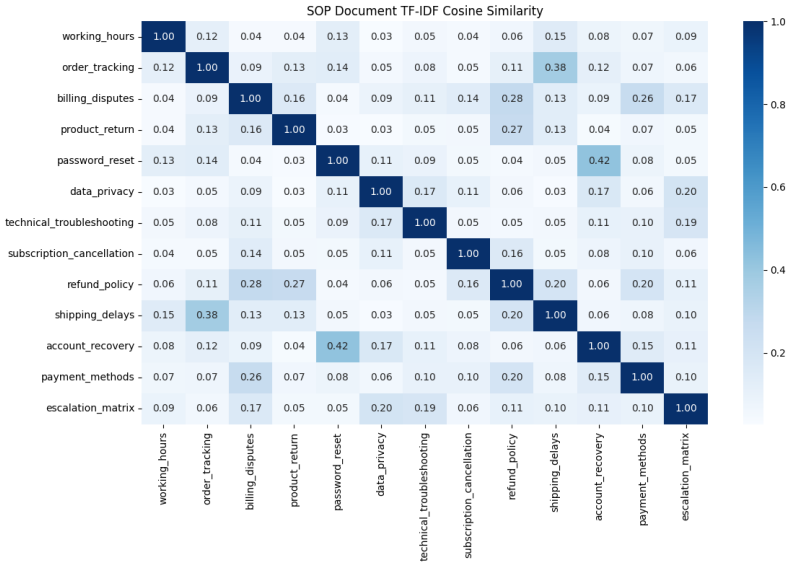


Figure 3. TF-IDF cosine similarity between the 13 SOP documents, from the EDA notebook.

3. Methodology (Task 1.4)

3.1 Baseline model selection (Task 1.4.1)

Selected model: Qwen/Qwen2.5-1.5B-Instruct, used in 4-bit NF4 quantisation by every system.

- **Hardware.** The development laptop has 16 GB of RAM and integrated graphics without CUDA support, so the GPU stages run on a free Kaggle T4 (15 GB). A 1.5-billion-parameter model in 4-bit uses a small fraction of that memory.
- **Dataset characteristics.** Prompts are short (mean 10 tokens) and a full ChatML example fits in under 80 tokens, so no long context is needed.
- **Fine-tuning feasibility.** LoRA on q_proj and v_proj trains 2,179,072 of 1,545,893,376 parameters (0.141%), which fits a short run on the T4.
- **Fit for the task.** It is instruction-tuned with a native ChatML template, which suits structured JSON output, and it is the recommended model for this capstone. TinyLlama-1.1B is weaker at structured output and Llama-3-8B-Instruct is likely to exceed the free T4's memory.

3.2 Retrieval strategy (Task 1.4.2)

- **Embeddings.** sentence-transformers/all-MiniLM-L6-v2 (384 dimensions), run on CPU.
- **Vector store.** A persistent ChromaDB index of the 13 SOP documents, stored whole because each file is short and single-topic; the index count is validated against the number of files.
- **Search.** Similarity search returning the top document (k=1). Naive RAG searches with the raw customer query. Hybrid RAG searches with a string derived from the router's intent (underscores replaced by spaces and title case, for example track_order becomes “Track Order”), falling back to the raw query if the router output is not valid JSON.

- **Prompt augmentation.** The retrieved text is placed in the system prompt after “Answer strictly using this SOP:”.
- **Retrieval evaluation.** Manual inspection of the retrieved file names on validation queries, and indirect measurement through ROUGE, BLEU and the hallucination check against the SOP reference.

3.3 Fine-tuning strategy (Task 1.4.3)

- **Method.** LoRA on the 4-bit base model (NF4, double quantisation, fp16 compute) with r=16, alpha=32 and dropout 0.05, applied to q_proj and v_proj.
- **Data format.** ChatML through the Qwen chat template: a system prompt describing the intent classifier, the customer query as the user turn, and the strict JSON target as the assistant turn. Sequences are padded to 128 tokens.
- **Loss masking.** Padding positions are labelled −100, and so is every prompt token up to and including the assistant header (<|im_start|>assistant). The loss therefore covers only the JSON answer and its end-of-turn token. This goes one step beyond the notebook's Task 2.3.1 text, which describes padding-only masking; the task markdown was not updated, and the extension was made deliberately after a padding-only run left the router with poor label accuracy.
- **Training.** A custom PyTorch loop with AdamW, a constant learning rate of 2e-4, batch size 4 and a cap of 800 steps (about 3,200 rows, roughly one epoch). The model is validated on all 390 validation rows every 10 steps, with early stopping (patience 3).
- **Checkpointing and logging.** The best checkpoint by validation loss is saved as intent_lora_best and the final checkpoint as intent_lora. Train and validation losses are logged to training_log.csv and plotted as training_curves.png.
- **Reproducibility.** The model ID, quantisation and LoRA settings, learning rate, batch size and step limits are recorded. Sampling and splits use seed 42; the training loop itself is not seeded.

3.4 Hybrid RAG integration

- 1. The fine-tuned router (adapters merged into the base model) reads the raw query with the same system prompt used in training and emits a JSON intent (at most 30 new tokens).
- 2. The intent is mapped to a search string, with a fallback to the raw query if the JSON is invalid.
- 3. ChromaDB returns the top SOP for that search string.
- 4. The model generates the final response with the retrieved SOP in the system prompt. The final response is parsed and format-checked on its own, separately from the router output.

3.5 Evaluation framework (Task 1.4.4)

Metric	Procedure
Format adherence	Parse the output as JSON after stripping code fences. For V2 this is measured on the final response, and router adherence is reported as a separate figure. The live V1 notebook (Notebook 5) additionally requires non-empty intent and category fields.
Intent accuracy	Exact match between the predicted and Bitext intent. A fuzzy match (at least half of the true label's words, underscores treated as spaces) is reported alongside to catch word-order variants such as order_tracking and track_order. For V2 the intent is taken from the router.
ROUGE and BLEU	rouge-score (ROUGE-1 and ROUGE-L, with stemming) and NLTK sentence BLEU (smoothing method 1) against the top SOP retrieved for the raw query.
Consistency	Three greedy runs on five queries; the share of queries with identical outputs.
Hallucination	A response is flagged if it contains a timeline, “department X”, “call us at” or “tracking number X” that the reference SOP does not contain.
Evaluation data	The held-out test split (391 rows, never used in training or validation). Baseline, V1 and V2 are compared on all 391 rows. The live V1 evaluation in Notebook 5 uses a 50-row sample (seed 42). The adversarial subset is the set of test rows matching hedging or difficulty words (again, problem, issue, wrong, help, cannot, can't), which were extracted from the data because the workflow's suggested words matched none; 101 rows matched and all were evaluated.
Decoding	do_sample=False, temperature=None and top_p=None for every system, so results are deterministic.

4. Environment and technology stack

Area	Tools
Environment	Stages 1–2 on a local Jupyter notebook; Stages 3–4 on Kaggle with a T4 GPU
Data processing	pandas, numpy, scikit-learn, matplotlib, seaborn
Model development	transformers, datasets, torch, accelerate, peft, bitsandbytes
Retrieval	sentence-transformers, ChromaDB, LangChain document loaders
Evaluation	rouge-score, nltk, scikit-learn